

编译原理

2025 春

任课教师：柯嵩宇

2025-02-27

自我介绍

- 个人经历

- 2018年本科毕业于上海交通大学 ACM班
- 2024年上海交通大学博士毕业
- 2024年10月起加入福州大学计算机与大数据学院

- 写过编译器!!!

- 不容易，但也不难
- 挺花时间的

- 联系方式

- 邮箱: songyuke@fzu.edu.cn
- QQ: 604282122 (有问题优先发邮件, QQ信息太多容易漏)

对大家的期望

- 学术伦理
 - 作业代码禁止抄袭!!!
 - 抄袭项目0分!!!
- 积极的学习态度
 - 师傅领进门，修行靠个人
- 建议自学内容
 - 自动机理论：DFA、NFA、下推自动机、图灵机、P与NP
 - 计算机体系结构：流水线、分支预测、乱序执行
 - LLVM使用以及SSA中间表示
 - 寄存器分配算法

课程参考书

- 龙书
 - Compilers: Principles, Techniques, and Tools
 - 经典之一，成书较早，包含一些过时的技术
 - 目前出了第二版，包含了面向对象设计的一些新技术
- 虎书
 - Modern Compiler Implementation in Java
 - 后面出了C版本，但只有Java版本的封面是一只老虎
- 鲸书
 - Advanced Compiler Design and Implementation
 - 与“龙书”齐名，非常甚至是过于详细了
- 自制编译器, (日)青木峰郎

相关链接: <https://github.com/Jiar/Awesome-Fundamentals-Of-Compiling-Resource>

课前一件事

- 本课程的孪生课程——编译原理设计实践
 - 需要和本课程选择同一个任课教师
 - 本课程：103
 - 实践课程：97
 - 请同学们尽快处理一下

课前小调查

什么是编译程序？

- DEV-C++?
- Visual Studio?
- Xcode?
- Visual Studio Code/Sublime/Vim?
- Python?
- GCC、G++
- Clang

前置课程

- C语言、数据结构、算法
- 计算机体系结构/计算机组成原理
 - CPU如何工作
 - 流水线、分支预测、乱序执行
- 汇编语言
- 计算复杂性/自动机理论/.....
 - 有限状态自动机、下推自动机、图灵机

第一讲 概论

直击灵魂的问题

- 为什么要学这门课？

- 被强制的，这是必修课、培养计划要求
- 名字里带原理的听着就很厉害，一定很重要
- 编译原理是自主技术生态构建的基石
 - 通过编译技术对AI框架进行底层指令级优化，可显著提升模型训练和推理效率，减少对高端芯片的依赖
- 编译原理支撑国产软硬件的深度适配
 - 自主研发的LoongArch、MIPS、ARM、RISC-V等新硬件产品需要对应的软件生态
 - 软件的平台相关优化是必要的
- 编译原理助力应对技术安全挑战
 - Xcode被植入木马
 - 编译期安全检查（Rust）
 - 插桩用于性能测试、安全监控等
- 培养重要的抽象与系统化思维能力



另一个直击灵魂的问题

- 为什么需要编译器?
 - 回顾程序设计语言的发展历史

程序设计语言的发展历程

- 早期阶段
 - 机器语言：
 - 直接通过二进制指令控制硬件，编程效率极低且完全依赖硬件结构
 - MIPS指令结构 →
 - Intel x86 nop instruction: 0x90

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Type	Op	Rs	Rt	Rd	Shamt	Func
I-Type	Op	Rs	Rt	16 bit Address or Immediate		
J-Type	Op	26 bit Address (for Jump Instruction)				

- 汇编语言：
 - 用助记符来表示机器指令，使得程序的编写和阅读变得相对容易
 - MOV、ADD、SUB等等
 - 与机器语言基本一一对应
 - 通过汇编器将其转换为机器语言
 - 汇编器会帮忙计算跳转位置和处理一些复杂操作

程序设计语言的发展历程

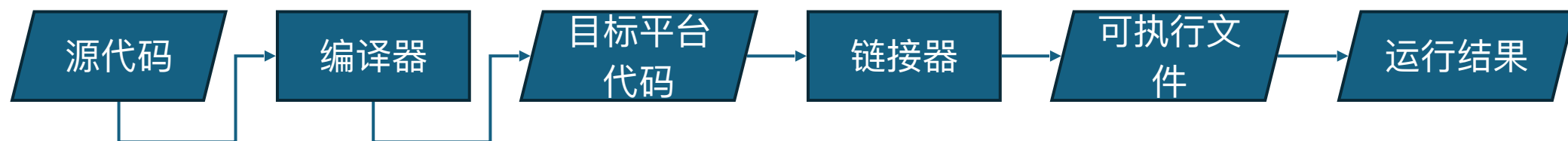
- 高级语言阶段

- 编程抽象化的飞跃
- 编译程序在此阶段承担了语义鸿沟的桥梁作用
 - 人类友好的循环（如for）、条件语句（如switch-case）转换为底层跳转指令；
 - 类型系统：支持在编译期完成类型检查（如Java强类型），避免运行异常；
 - 内存管理抽象：通过栈分配、垃圾回收（如Java语言）隐藏手动内存操作的复杂性。
- 例如，C语言编译器将 $a = b + c$ 转换为多条机器指令，自动处理数据移动和ALU操作。

程序设计语言的发展历程

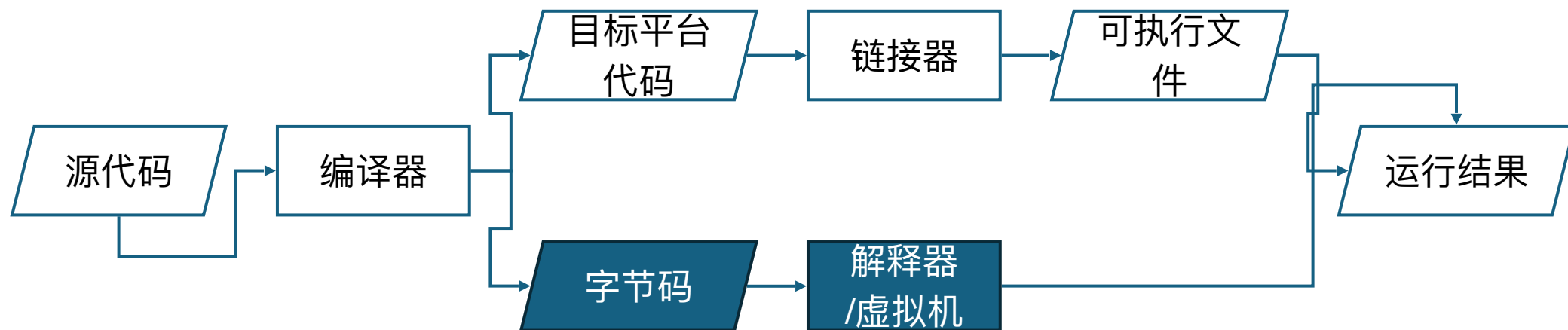
- 高级语言阶段
 - 代表语言：
 - C语言：
 - 几乎所有现代编程语言的鼻祖
 - Fortran：
 - 过程式编程、**数组按列存储**
 - LISP：
 - 函数式编程：代码即数据、函数也是一种类型
 - 前缀表示法：(+ 1 2 3 4)
 - Java、C#、Python：
 - 虚拟机语言
 - Rust
 - 标榜安全的编程语言
 - *LaTeX*
 - **SQL（特定领域语言，面向结果编程）（比高级语言更高级）**

计算机程序的执行流程



代表语言： C/C++、Pascal、Rust、Go

计算机程序的执行流程



代表语言：Java、Python、Javascript

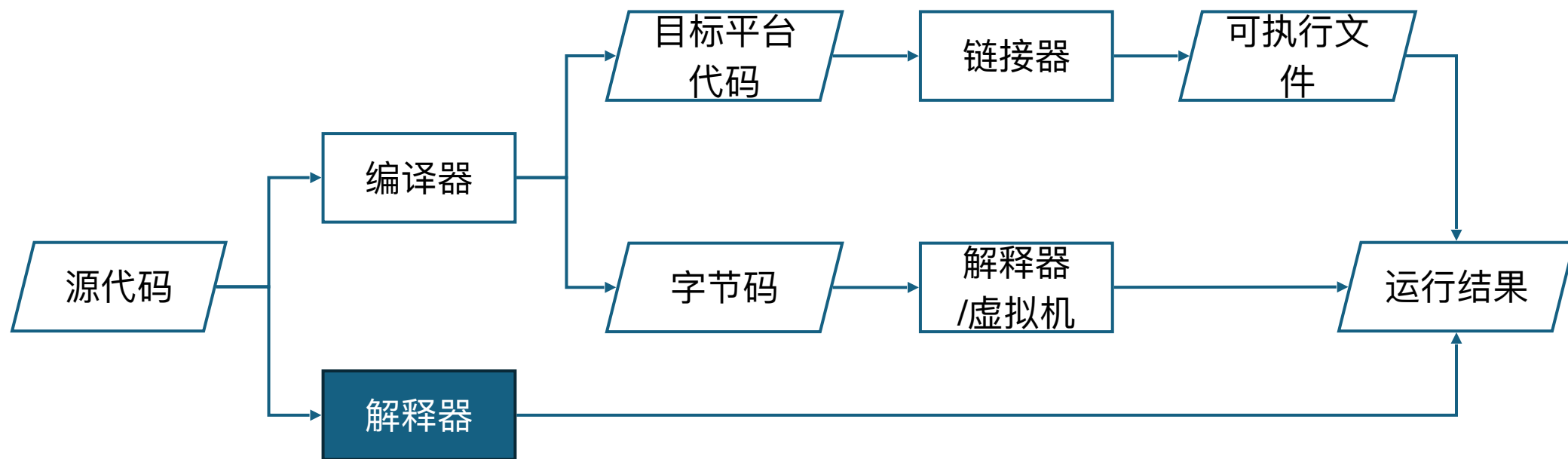
虚拟机(Virtual Machine)

- 寄存器
 - 32位x86平台有8个通用寄存器
 - 64位x86-64平台有16个通用寄存器
 - ARM平台有31个通用寄存器
 - MIPS32平台有32个通用寄存器和32个浮点寄存器
 - RISC-V平台同样有32个通用寄存器和32个浮点寄存器
- 虚拟机架构
 - 栈：早期Java虚拟机
 - 无限寄存器：Android等

虚拟机(Virtual Machine)

- 即时编译(Just-in-Time)
 - 虚拟机统计每段代码的执行频率，对于高频代码尝试编译成二进制代码，后续可以直接执行
 - E.g., JavaScript
- 提前编译(Ahead-of-Time)
 - 在程序安装或初次执行的时候进行编译，按照当前机器的配置进行二进制翻译并执行相关优化
 - Android、各种游戏的GPU渲染器管线代码

计算机程序的执行流程



代表语言： Shell、SQL等

可执行文件的产生——以C语言为例

- 可执行文件(Executable File)的生成通常包含下列步骤：
 - 预处理(Preprocessing, gcc -E)
 - 宏展开、条件编译、文件包含
 - 编译(Compilation, gcc -S)
 - 词法分析、语法分析、语义分析、中间代码生成、机器无关优化、机器相关优化、目标代码生成
 - 汇编(Assemble, gcc -c)
 - 二进制文件生成
 - 链接(Linking)
 - 填充符号表

扩展阅读：《程序员的自我修养——链接、装载与库》

可执行文件的产生——以C语言为例

- 可执行文件(Executable File)的生成通常包含下列步骤：
 - 预处理(Preprocessing, gcc -E)
 - 去掉所有注释 (//、/**/)
 - 处理宏定义、展开宏 (#define)
 - 条件编译 (#ifdef、#ifndef、#else、#endif)
 - 其他编译器指令#pragma

扩展阅读：《程序员的自我修养——链接、装载与库》

可执行文件的产生——以C语言为例

- 可执行文件(Executable File)的生成通常包含下列步骤：
 - 预处理(Preprocessing, gcc -E)
 - 编译(Compilation, gcc -S)
 - 词法分析
 - 语法分析
 - 语义分析
 - 中间表示生成
 - 机器无关优化
 - 机器相关优化
 - 目标代码生成

扩展阅读：《程序员的自我修养——链接、装载与库》

可执行文件的产生——以C语言为例

- 可执行文件(Executable File)的生成通常包含下列步骤：
 - 预处理(Preprocessing, gcc -E)
 - 宏展开、条件编译、文件包含
 - 编译(Compilation, gcc -S)
 - 汇编(Assemble, gcc -c)
 - 二进制文件生成
 - 代码段(.text): 保存硬件直接执行的指令
 - 数据段(.data、.rodata): 保存已初始化的所有变量的值
 - 未初始化数据段(.bss): 保存未初始化的变量, 通常只需要声明一个大小
 - 符号表(.sym): 有强符号和弱符号之分, 区别在于有没有对应的内存地址

扩展阅读: 《程序员的自我修养——链接、装载与库》

可执行文件的产生——以C语言为例

- 可执行文件(Executable File)的生成通常包含下列步骤：
 - 预处理(Preprocessing, gcc -E)
 - 宏展开、条件编译、文件包含
 - 编译(Compilation, gcc -S)
 - 汇编(Assemble, gcc -c)
 - 链接(Linking, gcc)
 - 拼接不同文件代码和数据段
 - 重新计算符号地址
 - 确定弱符号指向

扩展阅读：《程序员的自我修养——链接、装载与库》

GCC演示

思考题

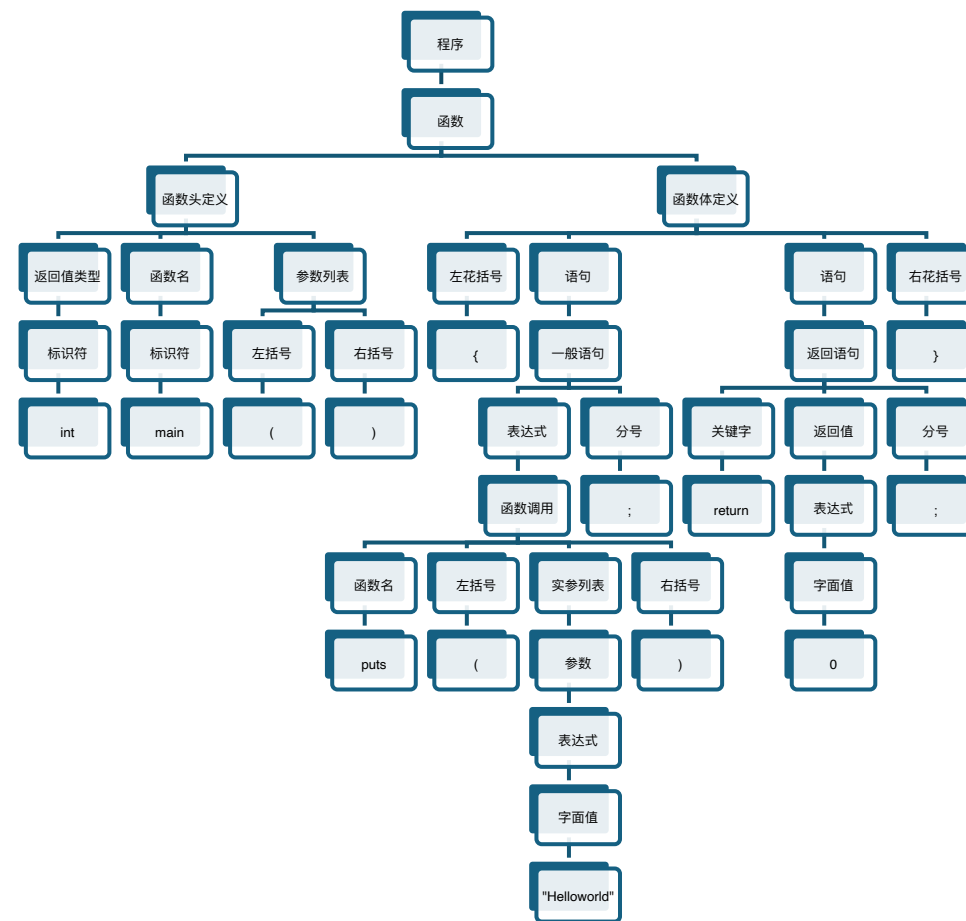
- 为什么C语言区分定义与声明？
 - 定义要分配内存地址
 - 声明只是一个占位符
- 为什么C语言讲究先声明再使用？
 - 简单的模块化设计
- 什么情况下会发生链接错误？
 - 全局变量找不到
 - 函数声明了但是没有定义

现代编译器的编译流程

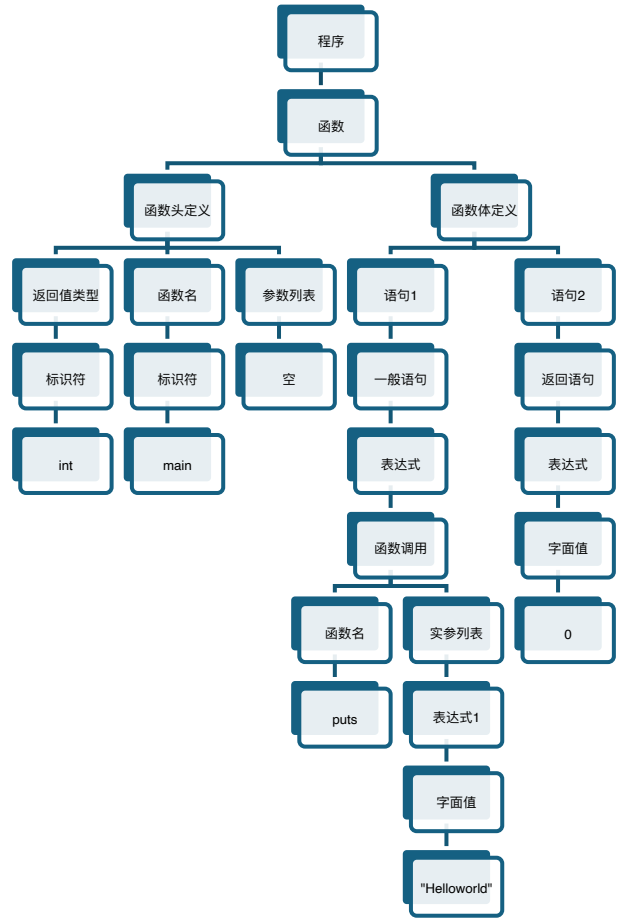
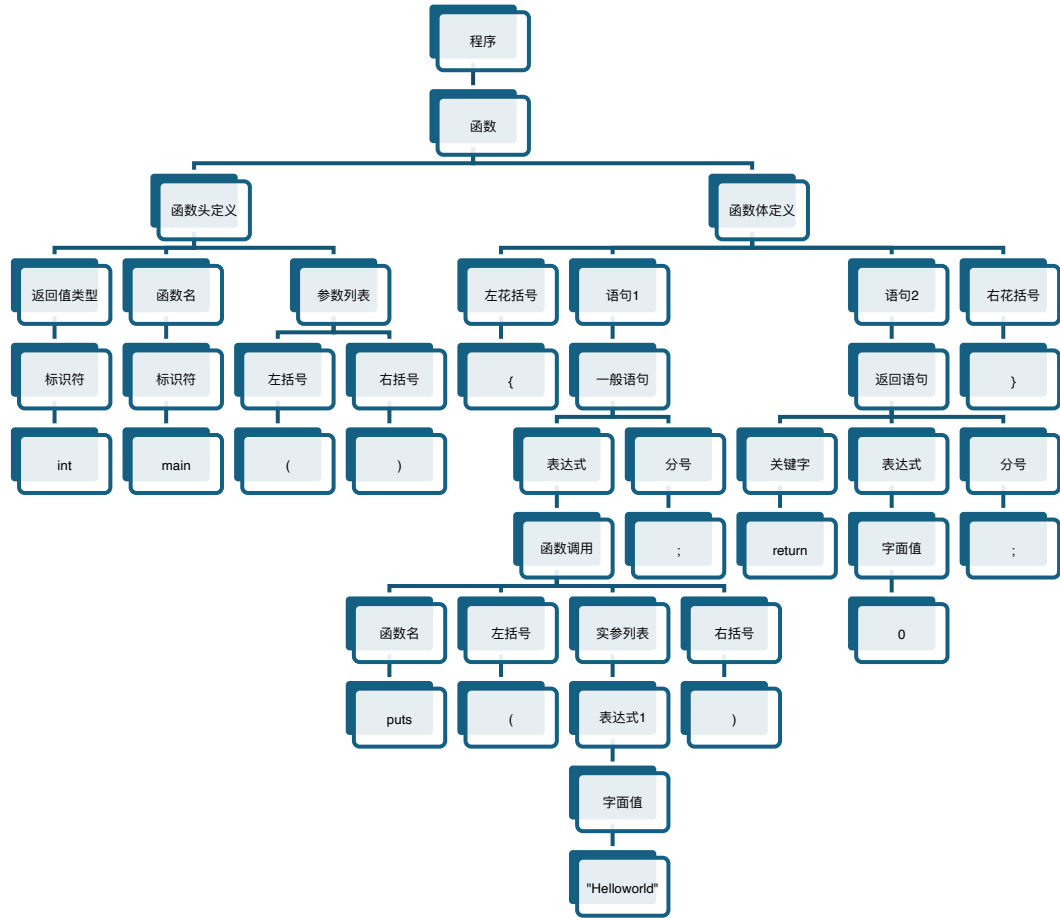
- 词法分析
 - 输入：字符流 (char stream) ，即源代码
 - 输出：词元流 (token stream)
 - 描述方法：正则表达式 (正则文法)
 - 处理工具：有限状态自动机
- 语法分析
 - 输入：词元流
 - 输出：语法树 (Syntax Tree)
 - 描述方法：上下文无关文法
 - 处理工具：下推自动机 (抽象形式，实际处理LL或者LR方法)

语法树

```
1. int main()  
2. {  
3.     puts("Helloworld");  
4.     return 0;  
5. }
```



具体vs抽象



具体vs抽象

具体语法树

- 保留原始程序的所有细节
 - 语句结构符号，比如括号、分号
- 直接遍历可以得到原始程序
- 有很多冗余的叶子节点

抽象语法树

- 简洁凝练
- 没有冗余的节点
- 良好的结构抽象
 - 表达式
 - 加法表达式、减法表达式、乘法表达式、除法表达式
 - 语句
 - 条件语句
 - 分支、开关
 - 循环语句
 - For
 - While
 - Do-while

编译流程

- 语义分析（语义检查）
 - 输入：（具体）语法树
 - 输出：（抽象）语法树
 - 目的：
 - 1.检查程序逻辑，符合语言定义，没有二义性；
 - 2. 添加必要中间过程（如推断数据类型、添加隐式类型转换等）
 - 例子：字符串+数值、整数加减浮点数

编译流程

- 中间表示生成 (Intermediate Representation)
 - 输入: AST
 - 输出: IR
 - 目的: 去除计算机执行无关的语义信息, 比如类的访问控制
- 平台无关优化
 - 输入: IR
 - 输出: Optimized IR
 - 目的: 常量压缩、常量函数求值 (constexpr)、消除无用代码、函数内联等
- 平台相关代码优化
 - 输入: IR
 - 输出: Optimized IR
 - 目的: 展开循环、分配寄存器
- 代码生成
 - 输入: IR
 - 输出: 汇编指令、字节码等
 - 目的: 指令选择、SIMD优化等

链接可执行文件

静态链接

- 代码中写死跳转位置，运行速度快

动态链接

- 编译器维护需要跳转的函数列表
- 由动态链接器对实际跳转位置进行填充
- 通常是库内存起始位置+偏移量
 - 程序：我需要libc库的puts函数，把函数地址写到0xabcdef00
 - 链接器：libc库的起始位置是0x12340000，puts函数相对偏移量是0x5678，计算得出puts函数实际内存地址是0x12345678，把0x12345678写入到地址0xabcdef00

链接可执行文件

静态链接

- 优势
 - 加载运行速度快
 - 无需运行时文件，同样系统平台可以直接运行
- 劣势
 - 可执行文件大
 - 函数已写死，难以更新
 - 基础函数副本多，占用内存空间大

动态链接

- 优势
 - 共享操作系统运行时提供的基础函数，空间占用小
 - 容易更新（更新对应so文件即可）
- 劣势
 - 加载时间长、额外的跳转指令
 - 动态链接库损坏或版本不匹配时无法运行

思考题

- 编译器一定是从源代码到汇编语言吗？
- 解释器与编译原理有关系吗？

开源编译器介绍

- GCC – GNU Compiler Collections
 - 由GNU计划支持的开源编译器集合
 - gcc(C)、g++(C++)、gcj(Java)、gfortran
- LLVM系列
 - 使用LLVM IR作为中间表示
 - 替代了新语言设计开发过程中C语言的地位
 - LLVM IR带有一定的高级语言特性，如类型注解等便于后续优化
 - LLVM IR可以解释执行

思考题

- 如果某一天，世界上所有的编程语言工具（汇编器、编译器、解释器等）和它们的源代码都没了。此时，联合国让你领导全世界最优秀的一批程序员来重建它们，你需要多久？（假设这些程序员足够优秀，开发任何语言的汇编器、编译器都只需要7天）

现代程序的内存管理

- 操作系统负责管理什么内存？
- 编译器负责管理什么内存？
- 运行时环境如何管理内存？

作业

- 提交要求
 - 格式：PDF
 - 语言：中文
 - 建议使用LaTeX完成，使用XeLaTeX编译
 - 发送到任课老师邮箱：songyuke@fzu.edu.cn
 - 命名规则：
 - 主题：2025编译原理课程作业1-<学号>-姓名
 - 附件名：2025编译原理课程作业1-<学号>-姓名.pdf
 - 邮件正文：没啥要求，记得礼貌用语和署名（姓名+时间）
- 允许**适度使用**大模型
 - 如使用大模型需提供额外信息：
 - 1. 大模型名称、版本、运营平台等；
 - 2. 提供完整的对话流程；
 - 作者本人应当对生成的内容负责

作业1

- 使用C语言（不要用C++，太复杂了）解决一个需要使用二叉查找树（红黑树、AVL等）的OJ题目（可以从你们历史代码里面找）；
- 使用GCC编译器，分别用O1、O2、O3三个级别的优化开关进行编译，在汇编代码层面比较优化的效果
- 使用Clang-LLVM编译器，添加--emit-llvm参数，在LLVM IR层面比较O1、O2、O3的优化效果
- 撰写一个图文并茂的报告
- 一周内完成

现代语言开发工具-ANTLR